

Base R Graphics — devices & par() cheatsheet

Data Visualization with R · viz-base.rsquaredacademy.com · CC BY-NC-SA 4.0

DEVICES

Devices are a stack. Opening one makes it active; closing it returns to the one underneath. Every open device must be closed or output lands somewhere you did not choose — usually `Rplots.pdf`.

<code>png(f, width, height)</code>	raster, units defaults to "px"; res is ignored unless units is physical
<code>pdf(f, width, height)</code>	vector, for print, width/height are inches
<code>svg(f, width, height)</code>	vector, for web. Needs a cairo build
<code>jpeg(f, ...) / tiff(...)</code>	raster, lossy / lossless
<code>dev.off()</code>	close the newest device; returns its name and number
<code>dev.cur()</code> / <code>dev.list()</code>	which device is active / which are open
<code>dev.new()</code>	push a copy of the current device onto the stack
<code>capabilities("cairo")</code>	TRUE if <code>svg()</code> will work

```
# dev.off() belongs INSIDE the loop, not after it
for (g in groups) {
  png(file.path(out, paste0(g, ".png")), width = 1800, res = 150)
  plot(x[[g]], y[[g]])
  dev.off()
}
```

PAR() — MARGINS AND TEXT

`mar` and `oma` are measured in **lines of text**, so they scale with `cex` and `pointsize`. A margin tuned on screen is wrong in a high-res export unless you raise `pointsize` too.

<code>par(mar = c(b, l, t, r))</code>	plot margins, in lines of text
<code>par(oma = c(b, l, t, r))</code>	outer margins, only for multi-panel figures
<code>par(mgp = c(3, 1, 0))</code>	margin lines, tick labels, axis title
<code>par(las = 0, 3)</code>	tick labels: parallel, horizontal, perpendicular, vertical
<code>par(bty = "n")</code>	box: "o", "r", "7", "u", " ", " ", "f", "i"
<code>par(xaxs = "i")</code>	"r" (default, 4% padding) or "i" (exact data range)
<code>par(yaxs = "i")</code>	same, for the y axis
<code>par(tck = -0.02)</code>	tick length; negative draws outside, 0 removes
<code>par(cex = 1.2)</code>	scale all plot text
<code>par(family = "mono")</code>	base font family
<code>par(font.main = 2)</code>	point size, main text

`las = 1` is worth setting on any chart with numeric ticks — it stops y-axis labels being drawn slanted.

PAR() — PANELS AND STATE

<code>par(mfrow = c(2, 2))</code>	panels filled by row
<code>par(mfcol = c(2, 2))</code>	panels filled by column
<code>layout(matrix, widths, heights)</code>	tagged panels; widths/heights in 0–1
<code>layout(1)</code>	cancel <code>layout()</code> , back to a single panel
<code>par(new = TRUE)</code>	draw onto the current plot without clearing it
<code>par(no.readonly = TRUE)</code>	capture all current settings, for restoring
<code>op <- par(x) / par(op)</code>	set and capture in one call, then restore

`par()` is device state and persists until changed. Capture and restore rather than hand-writing old values back — and in a function, wrap the restore in `on.exit()` so an error cannot leave the device mangled.

```
init <- par(no.readonly = TRUE) # snapshot everything
par(mfrow = c(1, 2), mar = c(4, 4, 3, 1))
plot(x1)
plot(x2)
par(init) # always restore
```

EXPORT RECIPES

<code>units = "in", res = 300</code>	the combination to use for anything printed
<code>pdf(onefile = FALSE)</code>	one file per page — safer for long batch jobs
<code>pdf(width = 6, height = 4)</code>	inches, not pixels
<code>pointsize = 12</code>	raise it on small or high-res exports, or labels vanish
<code>file.info(f)\$size</code>	print it in a batch loop; 0 means <code>dev.off()</code> was missed

COLOURBLIND-SAFE PALETTES (R >= 4.1)

```
palette.pals() # every built-in palette
palette.colors(palette = "Okabe-Ito") # 9 colours, colourblind-safe
palette.colors(palette = "Okabe-Ito", alpha = 0.5)
```

Use the hex values as **R prints them** — hand-typing them from memory is how palettes get subtly wrong. The first Okabe-Ito colour is black, so drop it with `[-1]` unless you want black as your first series.

```
# greyscale check: Rec. 601 luma, not the mean of the channels
rgb <- t(col2rgb(pal)) / 255
luma <- rgb %*% c(0.299, 0.587, 0.114)
grey <- rgb(luma, luma, luma)
```

If the greyscale version has series you cannot tell apart, colour was doing work it should not do alone. Add a second channel: `pch`, `lty`, or direct labels.

AXIS() — DRAWING YOUR OWN

<code>xaxt = "n" / yaxt = "n"</code>	suppress the automatic axis first , or you draw two
<code>axis(1, at, labels)</code>	1 bottom · 2 left · 3 top · 4 right
<code>axis(1, at, labels = FALSE)</code>	ticks with no labels
<code>axis(4)</code>	add a second, right-hand axis — usually a different unit
<code>axis(..., las = 1)</code>	force horizontal labels, e.g. for dates
<code>outer = TRUE</code>	draw on the figure margin, not the plot region

Axis positions are plain numbers, so dates work by converting and formatting.

```
when <- as.Date("2026-01-01") + 0:6 * 30
plot(when, sales, type = "b", xaxt = "n")
axis(1, at = as.numeric(when), labels = format(when, "%b %d"), las = 1)
```

DRAWING PRIMITIVES

<code>plot(x, y, type=)</code>	"p" points · "l" line · "b" both · "o" overplot · "h" histogram · "n" axes only
<code>points()</code> / <code>lines()</code>	add to the current plot without clearing it
<code>segments(x0,y0,x1,y1)</code>	straight segments: error bars, caps, reference lines
<code>arrows(x0,y0,x1,y1)</code>	line with a head; args run from → to
<code>polygon(x, y, col=)</code>	closed filled shape; CI bands via <code>c(x, rev(x))</code>
<code>rect()</code> / <code>text()</code>	boxes and labels at coordinates
<code>legend(x, y, legend)</code>	x/y coords, or "topleft", "bottomright", ...
<code>text(x, y, labels)</code>	labels at coordinates; pos, offset, padj
<code>mtext(text, side)</code>	margin text; side 1–4 as for <code>axis()</code>
<code>abline(a, b)</code>	reference line; also <code>v =</code> , <code>h =</code>

STATE YOU MUST RESTORE

Every call below changes device state that outlives the call itself. If you change it, restore it — or the next plot in the session silently inherits it.

<code>call</code>	leaves behind
<code>par(mfrow / mfcol)</code>	multi-panel layout; <code>layout(1)</code> cancels it
<code>par(mar/oma/mgp/las/bty)</code>	page geometry, persists until reset
<code>par(new = TRUE)</code>	a later <code>plot()</code> draws on top instead of clearing
an open device	output goes to the wrong place; <code>dev.off()</code> it
<code>setwd()</code>	inside a script, the working directory stays changed
<code>options()</code>	process-wide until changed back

Full explanations: Chapter 12, **Production Export & Graphical Devices** · viz-base.rsquaredacademy.com